

# Toward Channel-Robust and Receiver-Independent Radio Frequency Fingerprint Identification

Jie Ma<sup>1</sup>, Student Member, IEEE, Junqing Zhang<sup>1</sup>, Senior Member, IEEE, Guanxiong Shen<sup>2</sup>, Member, IEEE, Linning Peng<sup>1</sup>, Member, IEEE, and Alan Marshall<sup>1</sup>, Life Senior Member, IEEE

**Abstract**—Radio frequency fingerprint identification (RFFI) is an emerging method for authenticating Internet of Things (IoT) devices. RFFI exploits the intrinsic and unique hardware imperfections for classifying IoT devices. Deep learning-based RFFI has shown excellent performance. However, there are still remaining research challenges, such as limited public training datasets as well as impacts of channel and receiver effects. In this paper, we proposed a three-stage RFFI approach involving contrastive learning-enhanced pretraining, Siamese network-based classification network training, and inference. Specifically, we employed spectrogram as signal representation to decouple the transmitter impairments from channel effects and receiver impairments. We proposed an unsupervised contrastive learning method to pretrain a channel-robust RFF extractor. In addition, the Siamese network-based scheme is enhanced by data augmentation and contrastive loss, which is capable of jointly mitigating the effects of channel and receiver impairments. We carried out a comprehensive experimental evaluation using three public LoRa datasets and one self-collected LoRa dataset. The results demonstrated that our approach can effectively and simultaneously mitigate the effects of channel and receiver impairments. We also showed that pretraining can significantly reduce the required amount of the fine-tuning data. Our proposed approach achieved an accuracy of over 90% in dynamic non-line-of-sight (NLOS) scenarios when there are only 20 packets per device.

**Index Terms**—Contrastive learning, device authentication, Internet of Things, radio frequency fingerprint identification, Siamese network, unsupervised learning.

## I. INTRODUCTION

THE increasing use of wireless devices has raised security concerns regarding device authentication in the

Internet of Things (IoT) [2]. Conventional authentication methods rely on cryptographic algorithms combined with software addresses. However, cryptographic approaches rely on complicated mathematical algorithms, which tend to be computationally expensive. Many IoT devices are only equipped with limited processing capability and cannot afford costly cryptographic schemes [3]. Moreover, software addresses, e.g., media access control (MAC) addresses, can be easily tampered with. Therefore, there is a strong need for a low-complexity and reliable authentication scheme that is applicable to low cost IoT devices.

Radio frequency fingerprint identification (RFFI) is a lightweight physical layer-based device authentication technique [2]. It identifies wireless devices by analyzing the unique distortion of the received waveform caused by the imperfections of hardware components. The imperfection comes from the transmitter chain of the wireless device, such as mixers, oscillators, power amplifiers, etc [4]. These components deviate slightly from their nominal specifications due to the inevitable manufacturing process variations. It manifests as hardware imperfections such as I/Q imbalance, carrier frequency offset (CFO), and power amplifier non-linearity [5], [6], [7], [8], [9], [10], [11], [12].

An RFFI system typically involves a few transmitters, also known as devices under test (DUTs), and one or multiple receivers. A receiver will capture the physical layer waveform with the hardware impairments embedded. It will then predict the device identity based on the wireless signals by extracting unique features, using an RFFI engine. Deep learning has been the core of the RFFI engine to extract the discriminative features from interrelated hardware impairments [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26]. In the last decade, deep learning has demonstrated excellent capabilities in feature extraction and classification, leading to significant progress in image processing, natural language processing, etc [27]. However, RFFI tackles RF signals, which are different from images, videos, or text [28]. There are unique challenges to deep learning-based RFFI, such as limited public datasets [29], varying channel effects [30], [31], and the hardware impairments of the receiver [32], [33], etc.

Deep learning training is data-hungry, which requires a large amount of labelled training datasets. Acquiring a sufficient amount of training signals requires substantial collection and labour costs. It is common to fine-tune a pretrained model

Received 12 March 2025; revised 28 July 2025 and 25 October 2025; accepted 30 October 2025. Date of publication 7 November 2025; date of current version 14 November 2025. The work of Junqing Zhang was supported in part by U.K. Engineering and Physical Sciences Research Council (EPSRC) under Grant ID EP/V027697/1 and Grant EP/Y037197/1 and in part by U.K. Royal Society Research Grants under Grant RGS\R1\231435. The work of Linning Peng was supported in part by the National Natural Science Foundation of China under Grant 62171120. An earlier version of this paper was presented in part at the IEEE WCNC 2025 [DOI: 10.1109/WCNC61545.2025.10978330]. The associate editor coordinating the review of this article and approving it for publication was Dr. F. Javier Lopez-Martinez. (Corresponding authors: Junqing Zhang; Linning Peng.)

Jie Ma, Junqing Zhang, and Alan Marshall are with the School of Computer Science and Informatics, University of Liverpool, L69 3DR Liverpool, U.K. (e-mail: Jie.Ma@liverpool.ac.uk; Junqing.Zhang@liverpool.ac.uk; Alan.Marshall@liverpool.ac.uk).

Guanxiong Shen and Linning Peng are with the School of Cyber Science and Engineering, Southeast University, Nanjing 210096, China (e-mail: gxshen@seu.edu.cn; pengln@seu.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3630316

with a similar task in the deep learning community. However, most of the RFFI literature trains deep learning models from scratch [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], with a few exceptions [29], [34], [35]. Xu et al. [34] constructs a pretrained encoder using an unsupervised learning method, but overlooked several key factors, such as channels and receivers. Shen et al. [35] pretrained an RFF extractor in a supervised learning manner, but such a labelled training dataset may not always be available. The work in [29] adopted federated learning to train an RFF extractor among several edge receivers. However, distributed training may not be necessary when centralized training can be done offline, which will be less complicated.

The varying channel effects will impact the RFF extraction because the multipath will distort the received signals [36]. It is very common that RFFI will be subject to various channel effects, especially in mobile IoT systems, which calls for channel mitigation approaches. Such approaches can be categorized into signal processing-based methods and deep learning-based methods. Signal processing-based methods carefully design algorithms to eliminate channel effects, but preserve hardware impairments. For example, the work in [35] proposed channel independent spectrogram, which mitigates the channel effect by dividing the adjacent columns of the spectrogram. Channel equalization is also adopted for RFFI [36]. However, these algorithms may accidentally erase some hardware impairments. In addition, designing such algorithms require expert understanding on the signal features, which is not trivial. Regarding deep learning-based methods, data augmentation is widely employed [35], [37]. Specifically, data augmentation enriches the channel effects by simulation in the training stage, to make the training dataset as generalizable as possible. While the effectiveness of data augmentation has been demonstrated, it is not possible to emulate all the possible channel effects that will occur in the test stage.

The receiver hardware impairments also affect the RFFI performance, when multiple receivers are involved and/or different receivers are used in the deep learning training and test [32], [33], [38], [39], [40]. The work in [38] incorporates adversarial training to learn receiver-independent features by utilizing the gradient reversal layer to mitigate the influence of receivers. Zhao et al. [33] employs generative adversarial networks (GANs) to address the receiver-related challenges. However, a substantial volume of labelled datasets is essential for developing the system, particularly in the first phase, involving over one million signals, along with 50 transmitters and 28 receivers. Zha et al. [39] proposes a cross-receiver RFF learning based on contrastive learning. However, the evaluation is limited as only two types of USRP software-defined radio (SDR) platforms were involved as receivers. In [32], a global domain adaptation approach based on adversarial training, along with a subdomain adaptation method leveraging Local Maximum Mean Discrepancy (LMMD), is introduced to facilitate the extraction of receiver-independent features. Zhang et al. [40] proposes a separability condition aimed at minimizing the classification error probability at a new receiver.

Although previous studies have addressed channel and receiver challenges in RFFI, their effects are mitigated separately. In practice, these effects are highly likely to exist in the same system, e.g., in a mobile IoT system with multiple receivers. Joint mitigation of channel and receiver effects is essential but currently missing.

This paper proposed a three-stage approach involving contrastive learning-based pretraining, Siamese network-based RFFI classification network training, and inference, in order to tackle the challenges posed by limited training datasets as well as channel and receiver effects. We first pretrained an RFF feature extractor using unsupervised contrastive learning. Then, we fine-tuned a Siamese network-based RFFI classification network to jointly mitigate channel and receiver variations. During the inference stage, the received samples are fed into the trained neural network for device identification. Extensive experimental evaluations have been carried out by using three public LoRa datasets [35], [38], [41]. Additionally, we constructed a testbed consisting of ten LoRa DUTs and six SDR platforms as receivers; we collected a dataset that includes dynamic channel variations and multiple receivers. We carried out comprehensive experimental evaluations to assess the performance of pretraining and classification. Our technical contributions are summarized as follows.

- We used the spectrogram as the signal representation, which decouples transmitter impairments, channel effects, and receiver impairments without resulting in information loss.
- We pretrained an RFF feature extractor using unsupervised contrastive learning and publicly available training datasets. Since it is unsupervised learning, the label space is not required, which allows us to freely choose public resources. Data augmentation is adopted to train a channel-robust feature extractor.
- We designed a Siamese network-based RFFI classification network, enhanced by data augmentation and contrastive loss. We specifically constructed two fine-tuning datasets from two receivers for all the DUTs. A pair of these two signals from the same DUT but different receivers were augmented by channel effects. The contrastive learning then jointly mitigates the channel and receiver effects.
- We carried out extensive experimental evaluations on three labelled datasets for training/fine-tuning and one unlabelled dataset for pretraining. The experiment results demonstrated that pretraining is effective, especially when the fine-tuning data is limited. In addition, the channel and receiver effects can also be eliminated effectively by the Siamese network-based RFFI.

In our previous work [1], we showed that the contrastive learning-based RFFI system is robust to channel variation. Moreover, compared with IQ signals, spectrogram, and channel-independent spectrogram, the spectrogram with contrastive learning showed a higher accuracy in both static and dynamic channels. This paper further extended our previous work by jointly solving the impacts of channel and receivers via a Siamese network with contrastive learning. In addition, we explored the importance of the pretrain model in scenarios

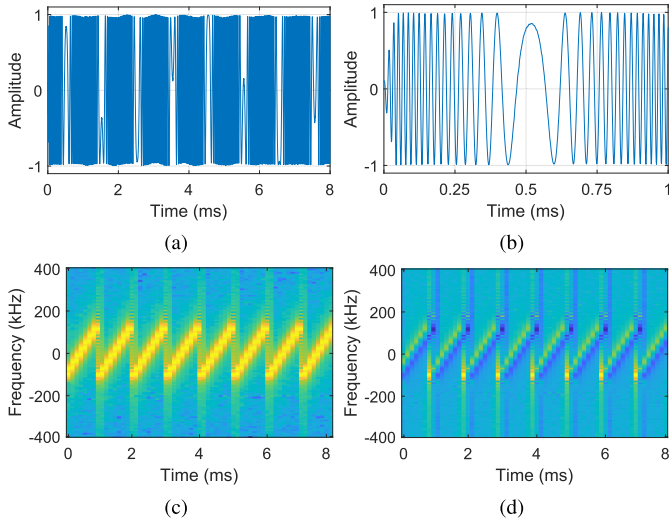


Fig. 1. (a) Time domain signal of the preambles (I branch). (b) Time domain signal of the first preamble (I branch). (c) Spectrogram of the preambles. (d) Channel-independent spectrogram of the preambles.

with a limited number of training samples. We also carried out a more comprehensive experimental evaluation.

The rest of the paper is organized as follows. Section II introduces the background and problem statement. Section III outlines our proposed system. Then, the detailed design methods of the Siamese-based RFFI and contrastive learning-based pretraining are elaborated in Sections IV and V, respectively. The experimental setup and involved datasets are introduced in Section VI. Sections VII and VIII present the experimental results for the classification and pretraining, respectively. Finally, Section IX concludes the paper.

## II. BACKGROUND AND PROBLEM STATEMENT

### A. LoRa Primer

LoRa is a widely used wireless technique for long-range IoT applications. LoRa employs chirp spread spectrum (CSS) modulation and the signal's instantaneous frequency varies continuously over time. There are several preambles at the beginning of each LoRa packet. A baseband LoRa preamble can be mathematically given as

$$s(t) = Ae^{j(-\pi Bt + \pi \frac{B}{T} t^2)} \quad (0 \leq t \leq T), \quad (1)$$

where  $A$ ,  $B$ ,  $T$  are the amplitude, bandwidth, and symbol duration, respectively.

The contents of the preambles are identical and fixed across all packets, making them ideal for RFFI. Fig. 1(a) shows the time-domain baseband signal of the preambles (I branch) and Fig. 1(b) zooms the first preamble.

### B. Deep Learning-Based RFFI

As shown in Fig. 2, a common deep learning-based RFFI system comprises  $K$  DUTs and a receiver. Based on the received signal, the receiver aims to predict the identity of the DUT. The deep learning-based approach consists of the training and inference stages.

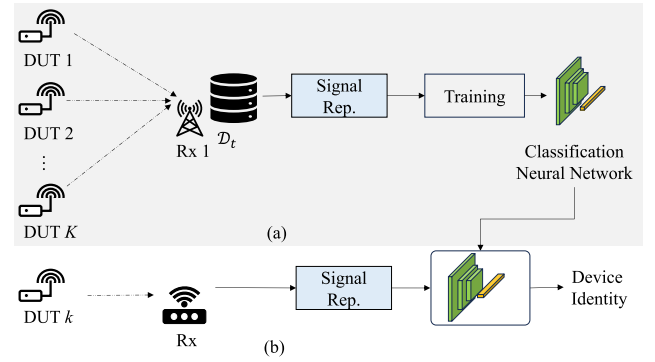


Fig. 2. Deep learning-based RFFI. (a) Training stage. (b) Inference stage.

- **Training stage:** A training dataset, i.e.,  $\mathcal{D}_t = \{(x_i, y_i)\}$ ,  $i = 1, \dots, KM$ , is constructed, where  $x_i$  and  $y_i$  represent the time domain signals (IQ samples) and its label (the index of the DUT), respectively,  $M$  is the number of samples from each DUT. The signal is first converted to a particular representation and then fed into a deep learning model for training. A trained model will be returned once the process is completed.
- **Inference stage:** The received signal undergoes the same signal representation procedure as that in the training stage. It is then input into the trained model for classification, and a prediction will be made.

The above process is general to deep learning-based RFFI. However, there are a few challenges in practical applications. This paper will focus on the channel and receiver effects as well as the requirement of labelled training data, which will be elaborated below.

### C. Channel and Receiver Effects

The received signal is expressed as

$$x(t) = \mathcal{G} * h(t) * \mathcal{F}_k(s(t)) + n(t), \quad (2)$$

where  $\mathcal{G}$  represents the effects of the hardware impairments of the receiver,  $h(t)$  is the wireless channel impulse response,  $*$  represents the convolutional operation,  $\mathcal{F}_k(\cdot)$  denotes the effect of the  $k^{\text{th}}$  DUT's transmitter impairments, and  $n(t)$  denotes additive white Gaussian noise (AWGN).

When RFFI is applied in a dynamic environment, the channel effect  $h(t)$  is time-varying. Regarding receivers, it is highly likely that different receivers are used in the training and inference stages, resulting in different receiver impairments. The channel effect  $h(t)$  and the receiver effect  $\mathcal{G}$  will change the data distribution of  $x(t)$  both in the training and inference stages, therefore, mitigating the channel and receiver effect is essential. However, it is quite challenging to eliminate the effects in the time domain due to the convolutional process.

### D. Requirement of Labelled Training Data

In order to train a deep learning model with a good generalization capability, sufficient samples from each DUT are required in the training dataset, which significantly increases the collection overhead and labour costs. In the computer

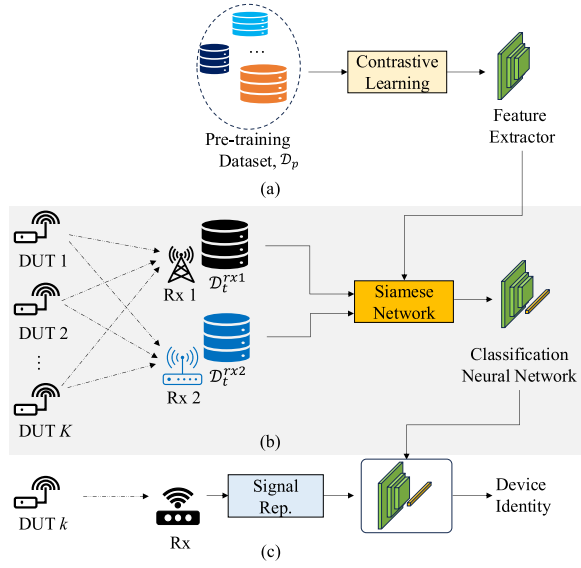


Fig. 3. The proposed three-stage RFFI approach. (a) Contrastive learning-enhanced pretraining stage. (b) Siamese network-based training/fine-tuning stage. (c) Inference stage.

vision community, it is common to fine-tune a pretrained deep learning model using a small number of samples. However, in the RFFI community, due to the lack of a comprehensive dataset, many researchers tend to collect their own training datasets and train the RFFI model from scratch.

There are indeed a few RFFI datasets publicly available [35], [38], [42], [43]. However, RFFI relies on the hardware impairments of DUTs, and it is unlikely the datasets would have the same DUTs. In other words, their label spaces are different.

### III. SYSTEM OVERVIEW

To address the above challenges, a three-stage RFFI approach is designed, including pretraining, fine-tuning, and inference, as shown in Fig. 3. Specifically, unsupervised contrastive learning is adopted in the pretraining stage to obtain a channel-robust RFF feature extractor. In the fine-tuning stage, a Siamese network enhanced with contrastive loss is employed to eliminate the effects of channel and receiver variations.

#### A. Pretraining

As shown in Fig. 3(a), the pretraining stage used contrastive learning to train a feature extractor. Firstly, the pretraining dataset,  $\mathcal{D}_p$ , can be created by combining public datasets. These datasets may involve different numbers of DUTs and receivers. Unsupervised learning is adopted in this stage, which allows us to leverage the collected signal samples without worrying about the reliability of their labels. Therefore, contrastive learning, which excels at capturing the representative features by treating the signal itself as the label, is employed. This approach is based on maximizing the similarity between two augmented versions of the same sample. The data augmentation here is used to simulate signals with various channel and noise effects, enabling the development of

a channel-independent feature extractor. After the pretraining is completed, a feature extractor  $\mathcal{T}(\cdot, \theta_{\mathcal{T}})$  will be returned, where  $\theta_{\mathcal{T}}$  is the learnable parameters of the feature extractor. The details of pretraining will be presented in Section V.

#### B. Fine-Tuning

As shown in Fig. 3(b), a Siamese network with two branches is adopted in the fine-tuning stage. While the Siamese network is usually designed for pairwise similarity comparison, this paper uses it differently for the multi-class classification task, which is achieved by using the cross-entropy loss together with the contrastive loss.

A neural network,  $\psi(\cdot, \theta_{\psi})$ , will be created by concatenating the pretrained feature extractor and a dense layer,  $\mathcal{C}(\cdot, \theta_{\mathcal{C}})$ , for classification, given as

$$\psi(\cdot, \theta_{\psi}) = \mathcal{T}(\cdot, \theta_{\mathcal{T}}) \circ \mathcal{C}(\cdot, \theta_{\mathcal{C}}), \quad (3)$$

where  $\theta_{\psi}$  and  $\theta_{\mathcal{C}}$  represent the learnable parameters of the neural network and classification head, respectively.

There are  $K$  DUTs to be classified in this work. To solve the receiver issue, we deliberately employ different receivers, with at least two receivers, to capture signals from these  $K$  DUTs to construct fine-tuning datasets, i.e.,  $\mathcal{D}_i^{rx1}$  and  $\mathcal{D}_i^{rx2}$ . We select packets from the same DUT but different receivers as a pair to send into the Siamese network. This approach facilitates obtaining receiver-independent features. In addition, data augmentation will be used to enhance the neural network to be channel independent.

After fine-tuning, a neural network  $\psi(\cdot, \theta_{\psi})$  that is receiver and channel independent will be obtained. The fine-tuning details, such as deep learning architecture and loss function, will be introduced in Section IV.

#### C. Inference

Siamese networks usually involve two branches for pairwise similarity comparison during the inference stage. We use it different by only using one branch for multi-class classification, as illustrated in Fig. 3(c).

First, a receiver acquires a packet from the DUT that needs to be identified. Second, the received signal is transformed into a representation, e.g., a spectrogram, which is then directly input into the neural network for classification. Finally, the prediction of the signal will be given as

$$\hat{P} = \psi(X, \theta_{\psi}), \quad (4)$$

where  $X$  is the signal representation, and  $\hat{P}$  is a label predicted by the model  $\psi(\cdot, \theta_{\psi})$  for the most likely DUT of the input.

### IV. TRAINING/FINE-TUNING RFFI USING SIAMESE NETWORK

The RFFI is sensitive to the variations of the wireless channel and the receivers. To address these challenges, we designed a Siamese network-based RFFI approach, as shown in Fig. 4. The model can be trained from scratch or fine-tuned from a pretrained model. The technical algorithm design of the above two schemes is the same.

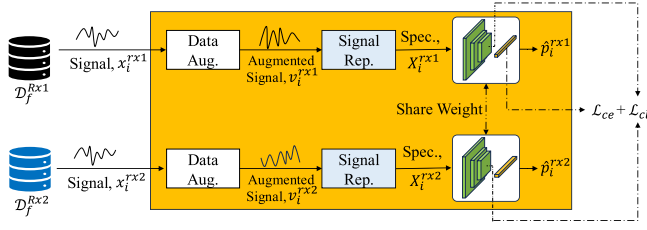


Fig. 4. The proposed Siamese network-based training/fine-tuning process.

### A. Overview

We first constructed a training dataset, consisting of two sub training datasets. Each sub-dataset includes the time domain signals (IQ samples) from the same set of DUTs but captured by different receivers, i.e.,  $\mathcal{D}_i^{rx1}$  and  $\mathcal{D}_i^{rx2}$ . Every time, we selected a signal from each subdataset,  $x_i^{rx1}$  and  $x_i^{rx2}$ , which have the same DUT label. In this paper, in order to reduce the overhead of collecting an abundant training dataset, we deliberately captured signals with high signal-to-noise ratio (SNR). Then, we used data augmentation to enrich the signals with channel variations and noise, which will be introduced in Section IV-B. Afterwards, the augmented signals will be transformed into a specific representation, namely spectrogram in this paper, in order to disentangle channel and receiver effects from the transmitter impairments, which will be presented in Section IV-C. The signal after representation is fed into the Siamese network model, which is composed of two identical subnetworks with shared weights and parameters. Enhanced by the contrastive loss, the Siamese network allows the model to effectively learn the similarities between input pairs through a common feature space, which will mitigate the channel and receiver effects. The overall loss function is formulated as the sum of a contrastive loss  $\mathcal{L}_{cl}$  and a cross-entropy loss  $\mathcal{L}_{ce}$ . The detailed architecture of the Siamese network, along with the involved losses, is presented in Section IV-D.

### B. Data Augmentation

Data augmentation serves as a crucial part of the system, as it enriches training datasets. Both channel and AWGN augmentation are used. For the channel model, the tapped delay line (TDL) channel has been chosen. The multipath effect is modelled using the exponential power delay profile (PDP), while the Doppler effect is characterized by the Jakes model. Additionally, the synthetic AWGN is incorporated in data augmentation. The multipath and AWGN augmentation of the system ensures the system is generalized and performs well under different scenarios. Regarding the ranges of the aforementioned terms, the RMS delay spread ranges from [5, 300] ns, Doppler frequency spans [0, 5] Hz, and SNR is [10, 40] dB. These configurations are designed to expose the model to diverse channel conditions and SNR levels during training, thereby improving its robustness and generalization in real-world scenarios. Please refer to [35] for more information.

The simulation parameters mentioned above are randomized for each signal. Online data augmentation [14] is utilized, which applies data augmentation during each epoch in training.

It contributes to increasing the training dataset and improving the model's generalization.

After data augmentation, the original signals,  $x_i^{rx1}$  and  $x_i^{rx2}$ , become  $v_i^{rx1}$  and  $v_i^{rx2}$ , respectively. Because they are from the same DUT, they later form a positive pair for the Siamese network. It is important to note that each sample in a training batch is independently augmented twice to form a positive pair. Moreover, random augmentation is applied at each training epoch, thereby exposing each signal instance to a diverse set of channel and noise conditions over time. This variability enables the model to learn robust and transmitter-specific representations that generalize effectively across a broad range of propagation environments.

### C. Signal Representation

Signal representation determines the neural network's input, which could be time-domain, frequency-domain, or other forms. In this paper, we employed the spectrogram, which reveals the time-frequency domain features, in order to separate the transmitter impairments from the receiver impairments and channel effects.

The time-domain IQ signal,  $x(t)$  in (2), can be transformed to the spectrogram in time-frequency domain,  $X(t, f)$ , by STFT, given as

$$\begin{aligned} X(t, f) &= \mathcal{S}(x(t)) \\ &= \mathcal{S}(\mathcal{G}) \mathcal{S}(h(t)) \mathcal{S}(\mathcal{F}_k(s(t))) + \mathcal{S}(n(t)), \end{aligned} \quad (5)$$

where  $\mathcal{S}(\cdot)$  represents STFT operation. We then convert the amplitude of the spectrogram from the linear scale to the logarithmic scale, which can be expressed as

$$\begin{aligned} \log |X(t, f)| &= \log |\mathcal{S}(\mathcal{G}) \mathcal{S}(h(t)) \mathcal{S}(\mathcal{F}_k(s(t))) + \mathcal{S}(n(t))| \\ &\approx \log |\mathcal{S}(\mathcal{G})| + \log |\mathcal{S}(h(t))| + \log |\mathcal{S}(\mathcal{F}_k(s(t)))|. \end{aligned} \quad (6)$$

The approximation holds when the signal has a high SNR level.

As shown in (2), the time domain signal involves both channel and receiver effects, which are difficult to separate due to the convolution operation. After it is converted to the spectrogram, the effects from the channel and receiver still exist. However, they become summation terms in the spectrogram, which are easier to separate.

After signal representation,  $x_i^{rx1}$  and  $x_i^{rx2}$  can be represented as follows:

$$x_i^{rx1} \rightarrow \log |\mathcal{S}(G_1)| + \log |\mathcal{S}(h_1(t))| + \log |\mathcal{S}(\mathcal{F}_k(s(t)))|, \quad (7)$$

$$x_i^{rx2} \rightarrow \log |\mathcal{S}(G_2)| + \log |\mathcal{S}(h_2(t))| + \log |\mathcal{S}(\mathcal{F}_k(s(t)))|, \quad (8)$$

where the terms highlighted in red are different between two signals, and the last term is common when  $x_i^{rx1}$  and  $x_i^{rx2}$  are from the same DUT.

### D. Siamese Neural Network

As shown in Fig. 4, different from a common deep learning network, a Siamese neural network involves two branches with the same architecture and shared weights.

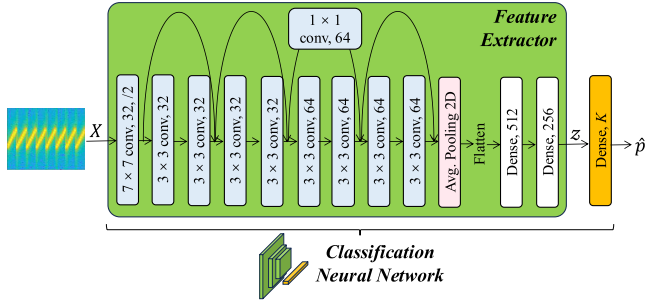


Fig. 5. The architecture of ResNet-based deep learning model consisting of a feature extractor and classifier.

1) *Deep Learning Architecture*: The deep learning model is portrayed in Fig. 5. As spectrograms can be regarded as 2D images, we designed the model based on ResNet, which is a popular convolutional neural network (CNN) architecture.

A classification CNN model can be further partitioned into a feature extractor part and a classifier.

- The *feature extractor* consists of nine convolutional layers, one average pooling layer, and two dense layers. The first convolutional layer uses 32  $7 \times 7$  kernels. The second to the fifth layers use 32  $3 \times 3$  kernels, while the sixth to the ninth layers employ 64  $3 \times 3$  kernels. Skip connections are employed to avoid gradient vanishing and facilitate optimization. The fully connected layers consist of 512 and 256 neurons, respectively. The output vector,  $z$ , serves as the RFF feature.
- The classifier consists of one dense layer with  $K$  neurons. It will make the prediction based on the RFF feature.

2) *Loss Function*: The Siamese network is optimized by the contrastive loss,  $\mathcal{L}_{cl}$ , and cross-entropy,  $\mathcal{L}_{ce}$ . The loss function  $\mathcal{L}$  is given as

$$\mathcal{L} = \mathcal{L}_{cl} + \mathcal{L}_{ce}. \quad (9)$$

The loss  $\mathcal{L}_{cl}$  and  $\mathcal{L}_{ce}$  is calculated by summing  $\ell_{i,j}$  and  $\ell_{ce}$  within a batch  $\mathcal{B}$ , respectively.

**Contrastive loss** plays a pivotal role in building the receiver and channel independent RFFI system. It enables the Siamese network to learn the common transmitter impairments and mitigate other effects between the samples in the two branches, which are from the same DUT but received by different receivers and augmented through distinct channels.

Contrastive loss maximizes the similarity of the positive pairs. A comparison between (7) and (8) reveals that the term  $(\log |\mathcal{S}(\mathcal{F}_k(s(t)))|)$  remains identical in both expressions for the positive pair, which represents the intrinsic characteristics of the device. Meanwhile, the effect of  $(\log |\mathcal{S}(\mathcal{G}_1)| + \log |\mathcal{S}(h_1(t))|)$  and  $(\log |\mathcal{S}(\mathcal{G}_2)| + \log |\mathcal{S}(h_2(t))|)$ , representing variations introduced by receivers and channels, will be mitigated. This ensures that the model learns the RFFs solely, without being influenced by other factors.

NT-Xent loss [27] is an effective and widely used contrastive loss, which was applied in our work. Given a set  $\{x_k\}$  including a positive pair of examples  $x_i$  and  $x_j$ , the contrastive loss helps to identify  $x_j$  in  $\{x_k\}_{k \neq i}$  for a given  $x_i$ . The loss function for

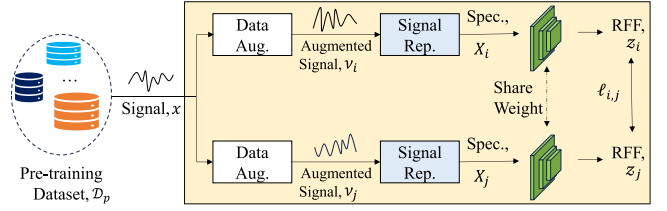


Fig. 6. Pretraining the feature extractor with contrastive learning.

each positive pair  $(i, j)$  is defined as

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j) / \tau)}{\sum_{k=1}^{2|\mathcal{B}|} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k) / \tau)}, \quad (10)$$

where  $\text{sim}(a, b) = a^T b / \|a\| \|b\|$  denotes the dot product between  $l_2$  normalized  $a$  and  $b$  (i.e., cosine similarity).  $\mathbb{1}_{[k \neq i]} \in \{0, 1\}$  is an indicator function that evaluates to 1 if  $k \neq i$ ,  $\tau$  denotes a temperature parameter (set as 0.05 in our work), and  $|\mathcal{B}|$  is the batch size. The similarity between RFF  $z_i$  and  $z_j$  will be maximized by (10), and the total loss  $\mathcal{L}_{cl}$  is computed across all positive pairs in a batch  $\mathcal{B}$ . In our implementation,  $|\mathcal{B}|$  is 32, each signal generates two augmented views, resulting in a total of 64 samples. Each sample serves as an anchor, with its counterpart forming the positive pair, and the remaining 62 samples acting as negatives for contrastive loss computation. All augmented samples are used as anchors in turn, thereby fully utilizing the pairwise relationships within each batch.

**Cross-entropy** assists the neural network in classification. The calculation is mathematically given as

$$\ell_{ce} = -\sum_{k=1}^K q_k \log(\hat{q}_k), \quad (11)$$

where  $q_k$  is the ground truth, and  $\hat{q}_k$  is the probabilities of the  $k^{\text{th}}$  label, returned by the softmax activation function.

## V. PRETRAINING FEATURE EXTRACTOR USING CONTRASTIVE LEARNING

While the deep learning approach presented in Section IV can work standalone, it indeed requires a large amount of labelled training dataset. It is common to pretrain the feature extractor  $\mathcal{T}(\cdot, \theta_T)$  and then use a few labelled samples to fine-tune the neural network.

The process of obtaining the feature extractor with contrastive learning in the pretraining stage is shown in Fig. 6. Although it has the same data augmentation and signal representation as in Fig. 4, the design philosophy is totally different. Specifically, the pretraining is done via unsupervised learning, while the training/fine-tuning in Section IV works in a supervised learning manner.

The pretraining dataset can be from a single source or from multiple sources. While the label information may not be reliable, we only use the signal samples, hence unsupervised learning is required.

A signal  $x$ , will be randomly selected from the pretraining dataset. It then will be individually augmented by adding random channel interferences and noise, via data augmentation. The process and parameter settings of the data augmentation

TABLE I  
SUMMARY OF EXPERIMENTAL EVALUATION

| Section | Evaluation Purpose                      | Pretrain | Dataset                       | Training Dataset                 | Test Dataset                                                 |
|---------|-----------------------------------------|----------|-------------------------------|----------------------------------|--------------------------------------------------------------|
| VII-A   | Channel elimination                     | N        | <i>Public Dataset 1</i>       | 10 DUTs<br>1 Rx (N210)           | Channel: LOS & NLOS and static & dynamic channels.           |
| VIII-B  | Pre-training, when only channel effect  | Y        |                               |                                  |                                                              |
| VII-A   | Channel elimination                     | N        | <i>Public Dataset 2</i>       | 10 DUTs<br>1 Rx (B210)           | Channel: Room & Outdoor & Office.                            |
| VII-B   | Receiver mitigation                     | N        | <i>Public Dataset 3</i>       | 10 DUTs<br>2 Rx (B200-1, B210-1) | Channel: LOS, high SNR.<br>Rx: 20                            |
| VIII-C  | Pre-training, when only receiver effect | Y        |                               |                                  |                                                              |
| VIII-C  | Joint channel and receiver mitigation   | N        | <i>Public Dataset 3</i>       | 10 DUTs<br>2 Rx (B200-1, B210-1) | Channel: NLOS and static (six locations).<br>Rx: 3 USRP N210 |
| VIII-D  | Joint channel and receiver mitigation   | Y        |                               |                                  |                                                              |
| VII-D   | Joint channel and receiver mitigation   | N        | <i>Self-Collected Dataset</i> | 10 DUTs<br>2 Rx (B210-a, N210-c) | Channel: LOS & NLOS and dynamic channels.<br>Rx: 6           |
| VIII-E  | Joint channel and receiver mitigation   | Y        |                               |                                  |                                                              |

are the same as Section IV-B. After data augmentation, each signal will become two copies,  $v_1$  and  $v_2$ .

After that, both copies are converted to the same signal representation used in Section IV-C, i.e., the spectrogram in this paper. The two signals then become

$$v_1 \rightarrow \log |S(G)| + \log |S(h_1(t))| + \log |S(\mathcal{F}_k(s(t)))|, \quad (12)$$

$$v_2 \rightarrow \log |S(G)| + \log |S(h_2(t))| + \log |S(\mathcal{F}_k(s(t)))|. \quad (13)$$

Because the signals are augmented from the same signal,  $x$ , only the channel parts are different between them.

Finally, the spectrogram is used to train the feature extractor. Because unsupervised learning is used, the loss function used in this stage is also the NT-Xent contrastive loss, which has been described in detail in Section IV-D.2.

Thanks to contrastive learning, it can alleviate effects from channels and obtain device-intrinsic features, which eventually produce a channel-independent feature extractor.

## VI. EXPERIMENTAL SETUP AND DATASETS

In order to evaluate the proposed approach comprehensively against various channel and receiver effects, we deliberately chose two public LoRa datasets [35], [38] and created our own dataset. The evaluations are summarized in Table I. It is noted that in this work, the training dataset consists of high-SNR samples with little channel and noise effects. Data augmentation applies various channels and SNRs to these high SNR samples, allowing us to simulate diverse conditions.

### A. Datasets

1) *Device Configuration*: All the datasets used in this paper follow the same device configurations. The LoRa DUTs were configured with bandwidth  $B = 125$  kHz. The sampling rate of the receiver SDR was  $f_s = 1$  MHz. The Matlab Communications Toolbox Support Package for SDR was used for accessing IQ samples from SDR platforms.<sup>1</sup> Eight preambles at the start of each LoRa packet were captured, as exemplified in Fig. 1(a).

<sup>1</sup><https://uk.mathworks.com/help/comm/supported-hardware-software-defined-radio.html>

2) *Public Dataset 1*: The work in [35] released a LoRa RFFI dataset, which includes test datasets collected in different locations. The DUTs were ten LoPy4 devices, and the receiver was a USRP N210 SDR platform.

a) *Training dataset*: There were 400 LoRa packets for each DUT, which were collected in a residential room with line-of-sight (LOS) available. The DUT and the receiver were kept close (about half a meter) to obtain high SNR data.

b) *Test dataset*: There were 200 packets per DUT. There were both static and dynamic channel scenarios, as well as LOS and non-line-of-sight (NLOS). The floorplan of the dataset collection position is shown in Fig. 7(a). The USRP N210 SDR platform was always placed at the red Rx position.

- Static channel: Test datasets D1-D6 were collected at six locations, labelled as A-F, respectively. The DUTs remained stationary, with no moving objects nearby.
- Dynamic channel: There were two cases. (i) Objects moving: D7 and D8 were collected at Location B and F, respectively, with the DUTs kept stationary while a person walked randomly around the room at a speed of 2 m/s. (ii) Mobile device: D9 and D10 were respectively collected in an office and meeting room, where a person walked around carrying the DUTs at a speed of 2 m/s.

Datasets D1, D2, D3, D7, and D9 were collected in LOS environments, while D4, D5, D6, D8, and D10 were collected in NLOS environments.

3) *Public Dataset 2*: The work in [41] released a LoRa RFFI dataset, which includes data collected in different locations, utilizing USRP B210 as the receiver and Pycom as the transmitters. For our evaluation, Dev 1 through Dev 10 were employed, and the preamble portion of each signal was extracted and utilized for RFF.

a) *Training dataset*: There were 400 LoRa packets for each DUT. All devices transmit an identical message from a fixed location, positioned 5 meters from the receiver, ensuring that each device experienced similar channel conditions. Data collection was conducted in an occupied indoor room.

b) *Test dataset*: There were 40 packets per DUT. LoRa transmissions were collected from three distinct deployment environments, i.e., an indoor room (Loc. 1), an office space (Loc. 2), and an outdoor area (Loc. 3), all on the same day.

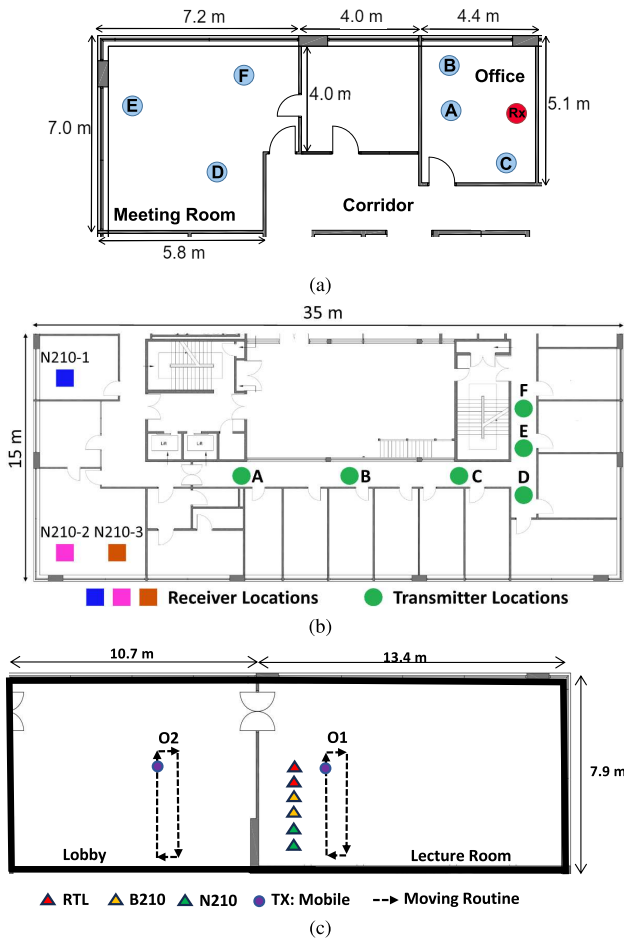


Fig. 7. Floor plan of the experimental environments. (a) *Public Dataset 1* [35]. (b) *Public Dataset 3* [38]. (c) Self-collected dataset.

4) *Public Dataset 3*: The work in [38] shared a LoRa RFFI dataset related to various receivers. The DUTs were ten LoRa devices, including five LoPy4 and five mbed 1261 shields. Various receiver SDR platforms were used.

a) *Training dataset*: 800 LoRa packets for each DUT-receiver pair were collected by B200-1 and B210-1, at an approximate distance of half a meter. The signals were with high SNR.

b) *Test dataset A*: For the receiver, twenty SDR platforms of six types were used, including nine RTL, two Pluto, two B200, two B200 mini, two B210, and three N210. There were 200 packets per DUT. All the devices remained stationary during data collection, located half a meter apart from the receiver. The signals were with high SNR.

c) *Test dataset B*: This test dataset involved different receivers and static channel effects. The involved receivers were N210-1/2/3 and the DUTs were placed in various locations. The floor plan of the experimental environment is given in Fig. 7(b). N210-1 was positioned in an office, while N210-2 and N210-3 were placed in a meeting room. The LoRa DUTs were tested at six locations, labelled as A to F. The SNR across Locations A to F decreased from 50 dB to 10 dB.

5) *Self-Collected Dataset*: The work in [38] only involved static channel effects, therefore, we created a dataset involving both different receivers and dynamic channel effects. As shown

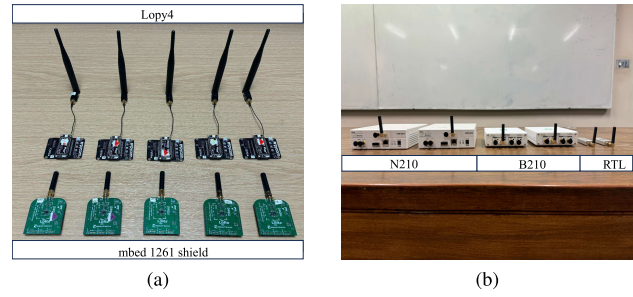


Fig. 8. Experimental devices. (a) DUT: ten LoRa devices. (b) Receiver: six SDR receivers.

in Fig. 8(a), five LoPy4 and five mbed 1261 shields were used as DUTs.

a) *Training dataset*: There were 800 LoRa packets for each DUT-receiver pair. During the data collection, the DUT and the receiver were placed stationary, one meter apart, with a LOS between them. The receivers used were B210-a and N210-c, where N210-c was not used in the test datasets.

b) *Test dataset*: Fig. 7(c) displays the floor plan of the environment where the test datasets were collected. The receivers involved consisted of two N210, two B210, and two RTL, labelled as N210-a/b, B210-a/b, and RTL-a/b as illustrated in Fig. 8(b). The test dataset includes 200 packets per DUT-receiver pair. The receiver was kept stationary while a person held a DUT and walked along routes O1 and O2 at a speed of 2 m/s for LOS and NLOS mobile scenarios, respectively.

## B. Benchmark Approaches

We employed the following benchmark algorithms for comparing the effects of channel mitigation, with different signal representations, e.g., channel independent spectrogram (CIS), spectrogram, and IQ samples, which are introduced as below.

- *CIS* [35]: This state-of-the-art method was designed to mitigate channel effects while preserving device-specific characteristics for reliable RFF. The representation is shown in Fig. 1(c).
- *CIS + Siamese*: CIS was used as the signal representation, and the deep learning model was trained with the Siamese network.
- *Spec*: Spectrogram was used as the signal representation.
- *IQ + Siamese*: IQ sample was as the signal representation, and the deep learning model was trained with the Siamese network.
- *IQ*: IQ sample was used as the signal representation.

For *CIS*, *Spec* and *IQ*, the deep learning was trained in the conventional way, i.e., the Siamese network was not involved. For *IQ*, the network architecture was also adapted from ResNet, as Fig. 5 shows, but the kernel size was adjusted according to the dimensions of the IQ samples.

When there were different receivers used, another two benchmark algorithms were also used.

- *CIS + Reverse* [38]: This state-of-the-art method adopted CIS as the signal representation and employed a gradient reversal layer to mitigate the influence of the receiver.

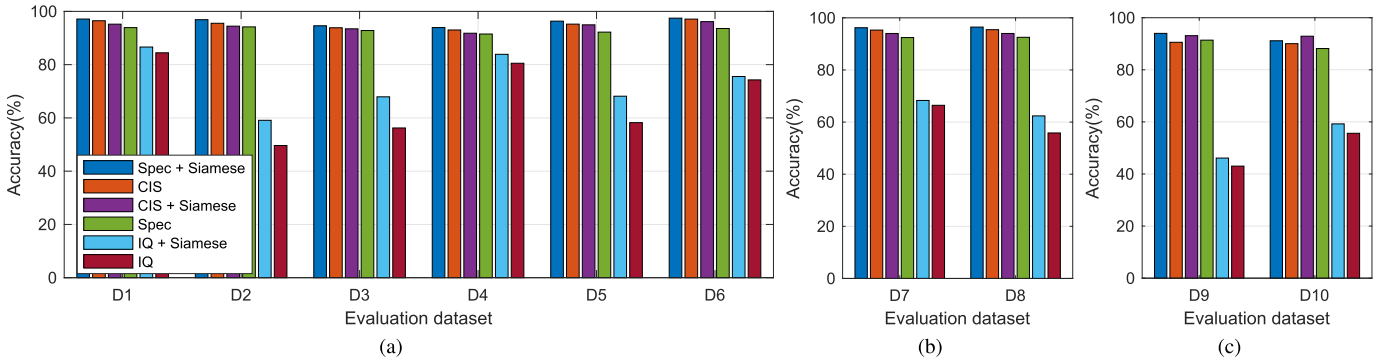


Fig. 9. Evaluation of the proposed Siamese network-based RFFI for the channel effect mitigation. *Public Dataset 1* [35] was used. (a) Test datasets D1-D6 were collected in static scenarios. (b) D7 and D8 were collected in dynamic scenarios involving moving objects. (c) D9 and D10 were collected in dynamic scenarios involving the moving DUT.

- *Spec + Reverse*: The spectrogram was used as the signal representation, and the gradient reversal layer was applied.

For the above two approaches, the Siamese network was not involved.

### C. Training Configuration

The deep learning training was carried out on a PC equipped with a GPU of NVIDIA GeForce RTX 4090, and PyTorch was used. The neural network parameters were optimized by the adaptive moment estimation (Adam) optimizer with an initial learning rate of 0.0003 and a batch size of 32. The validation loss was monitored during training, and the learning rate was reduced by a factor of 0.5 when the validation loss did not drop within 10 epochs. Training terminated when the validation loss did not change within 30 epochs.

### D. Metric

The classification accuracy is the primary evaluation criterion for the RFFI system, expressed as

$$\text{Accuracy} = \frac{\text{Number of correctly classified packets}}{\text{Total number of packets}}.$$

## VII. EXPERIMENTAL RESULTS OF CHANNEL AND RECEIVER MITIGATION

In this section, we conducted a comprehensive evaluation of the Siamese network's effectiveness in eliminating the influence of channel and receiver effects. There is no pretraining involved in this evaluation. Instead, we trained the Siamese network from scratch using the training datasets described in Section VI-A, which allowed us to study the effectiveness of Siamese network. In addition, we carried out ablation studies, by first investigating channel effect (Section VII-A) and receiver effect (Section VII-B) separately, and then combining both the channel and receiver effects (Sections VII-C and VII-D).

### A. Evaluation Against Channel Effect

The *Public Dataset 1* introduced in Section VI-A.2 only involved the channel effects because the same receiver was used for the training and test datasets.

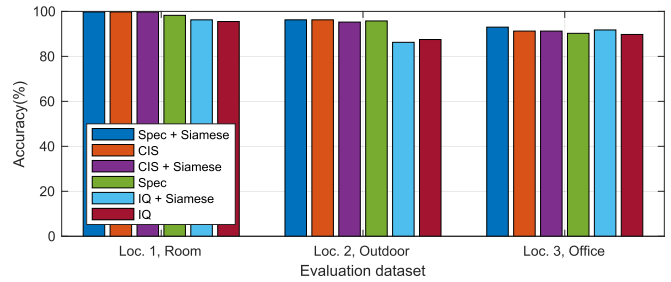


Fig. 10. Evaluation of the proposed Siamese network-based RFFI for the channel effect mitigation with the dataset published by *Public Dataset 2* [41].

The classification results in various channels are presented in Fig. 9. *Spec + Siamese* demonstrates the best performance across all test datasets. In static scenarios, the accuracy of the *Spec + Siamese* remains above 94% in LOS scenarios and 93% in NLOS scenarios. As for dynamic NLOS scenarios, its accuracy is still above 91%, which indicates that the proposed *Spec + Siamese* approach exhibits robustness across diverse channel conditions. When IQ samples were used, their performance was always worse than the spectrogram-based approaches, because the channel effects are convolved with the signal in the time domain and more difficult to eliminate.

The performance under dynamic channel conditions (D7 to D10) is worse compared to those under the static channels (D1 to D6), which is attributed to the increased multipath and Doppler effects in dynamic channels. The trend matched the results in [35]. It is worth mentioning that our *Spec + Siamese* has an accuracy of 91.2% in the most challenging D10 scenario. Furthermore, our *Spec + Siamese* approach always outperformed the *CIS* and *CIS + Siamese*. It is worth noting that both *CIS* and Siamese network designs aim to reduce the impact of channel effects. To perform a more comprehensive comparison, we additionally included the approach, *CIS + Siamese*, that combines these two methods. However, its overall performance is inferior to that of *Spec + Siamese*. This may be attributed to the use of *CIS* as the feature representation, since the division operations required in its computation could diminish hardware-relevant features.

To further validate the effectiveness of the proposed method, we conducted experiments on *Public Dataset 2*, and the corresponding results are shown in Fig. 10. The classification trends

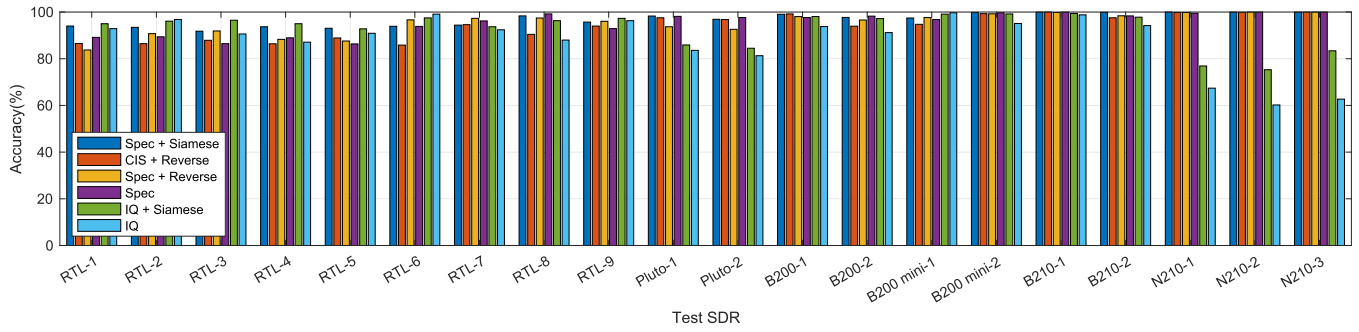


Fig. 11. Evaluation of the proposed Siamese network-based RFFI for the receiver effect mitigation. *Public Dataset 3* [38] was used. The training dataset was collected by B200-1 and B210-1. The **test dataset A** was used.

remained consistent with the former results, the proposed *Spec + Siamese* method achieved the best overall performance across all scenarios. In addition, IQ-based methods were observed to be less competitive.

### B. Evaluation Against Receiver Effect

Section VI-A.4 involves different receivers in the training and test stages. Only the **test dataset A** with high SNR signals is used in this evaluation, to focus on the receiver effects. All the twenty SDR receivers were used.

Fig. 11 depicts the classification accuracy of various receivers with different methods. The proposed *Spec + Siamese* achieves the best overall classification accuracy. The performance of *CIS + Reverse* and *Spec + Reverse* is slightly worse than that of *Spec + Siamese* in most cases. The occasional advantages are observed with the IQ-based methods, these occur in scenarios where all methods achieve relatively high classification accuracy, and the improvement brought by IQ-based methods is marginal. In contrast, in certain cases, the IQ-based method performs significantly worse than others, for instance, in scenarios involving receivers such as N210-1/2/3. As previously mentioned, when IQ data is used as input, wireless channel effects are embedded in the received signal in a convolved manner, making it difficult to eliminate. As a result, it is more challenging for IQ-based approaches to maintain consistently high performance.

### C. Evaluation Against Static Channel and Receiver Effects

The **test dataset B** of the *Public Dataset 3* presented in Section VI-A.4 is used in this section, which involves different receivers in the training and test datasets as well as static multipath channel effects. Different from the **test dataset A**, only three USRP N210 platforms were used in the **test dataset B**.

The classification results in various static channel environments are shown in Fig. 12. *Spec + Siamese* demonstrates the best performance in most cases. Nevertheless, it can be observed that in certain cases, such as position D, using IQ as the signal representation produced higher classification accuracies. This might be due to that the channel effects in these positions have been augmented during the training stage. However, the IQ-based approaches do not have a consistently high classification accuracy.

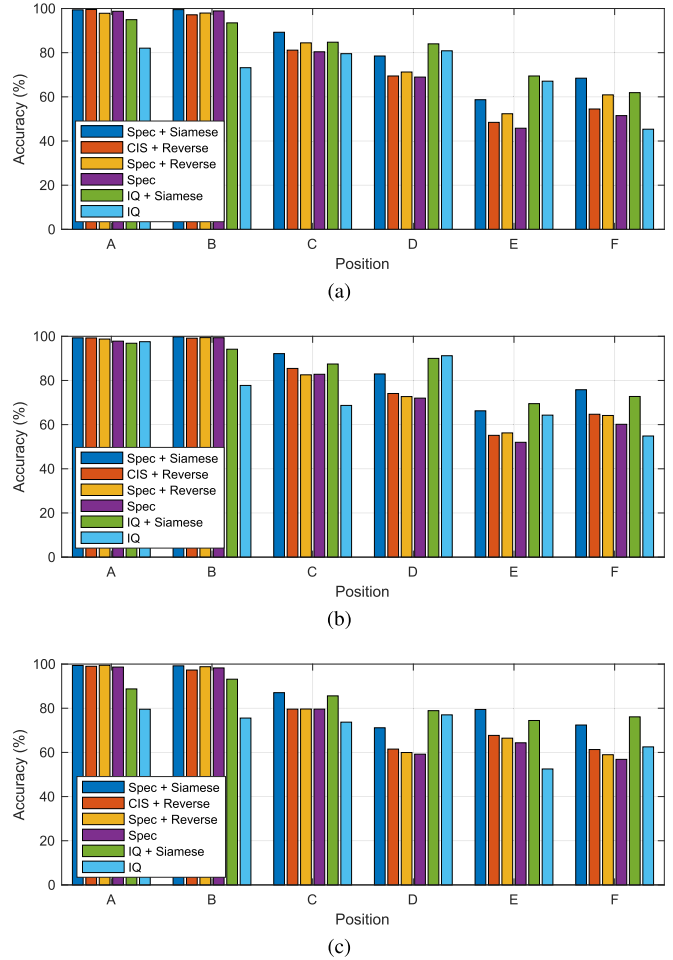


Fig. 12. Evaluation of the proposed Siamese network-based RFFI for the static channel and receiver effect mitigation. *Public Dataset 3* [38] was used. The training dataset was collected by B200-1 and B210-1. The **test dataset B** was used. The SNR of test datasets from A to F decreases from 50 to 10 dB. (a) Test dataset from N210-1. (b) Test dataset from N210-2. (c) Test dataset from N210-3.

### D. Evaluation Against Dynamic Channel and Receiver Effects

The dataset introduced in Section VI-A.5 is used in this section, which involves different receivers and dynamic channel effects. The receivers used in the training dataset were B210-a

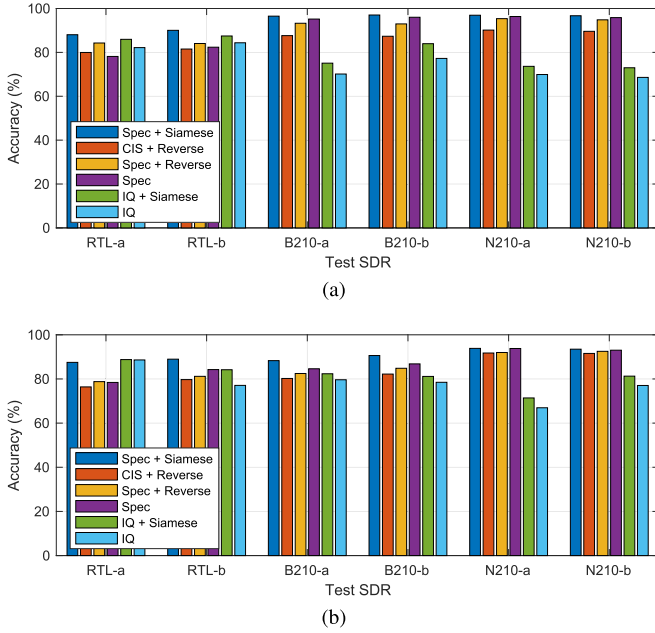


Fig. 13. Evaluation of the proposed Siamese network-based RFFI for the dynamic channel and receiver effect mitigation. Training dataset was collected by B210-a and N210-c. (a) Test dataset collected under LOS scenario. (b) Test dataset collected under NLOS scenario.

TABLE II  
COMPUTATIONAL COMPLEXITY AND INFERENCE TIME

| Method        | Inference Time (ms) | Parameter (M) | Storage (MB) | FLOPs (G) |
|---------------|---------------------|---------------|--------------|-----------|
| Spec+ Siamese | 1.13                | 13.94         | 53.19        | 0.58      |
| CIS + Siamese | 1.25                | 13.08         | 49.94        | 0.58      |
| CIS + Reverse | 1.28                | 13.09         | 49.94        | 0.58      |
| Spec+ Reverse | 1.16                | 13.94         | 53.19        | 0.58      |
| Spec          | 1.13                | 13.94         | 53.19        | 0.58      |
| IQ + Siamese  | 2.82                | 132.45        | 505.26       | 1.54      |
| IQ            | 2.82                | 132.45        | 505.26       | 1.54      |

and N210-c while the receivers used in the test dataset were RTL-a/b, B210-a/b and N210-a/b.

The classification accuracies are shown in Fig. 13. It can be found that the accuracy of *Spec + Siamese* is over 85% across all the receivers in both LOS and NLOS scenarios. Its performance surpasses other approaches and demonstrates greater stability. In contrast, IQ samples demonstrated unstable performance again. Fig. 13 shows that utilizing a spectrogram as input yields higher accuracy compared to using IQ as input in most cases, which aligns with the observation in Fig. 12.

#### E. Complexity and Inference Time Comparison

Table II provides a comprehensive comparison of computational complexity and inference efficiency across different methods on four key metrics: inference time, number of parameters, storage requirements, and floating point operations (FLOPs). Among all evaluated configurations, the methods with spectrogram or CIS show more efficient performance than IQ methods. The methods based on Spec or CIS maintain low inference times, approximately 1.13 to 1.28 milliseconds, and consistently require around 13 million parameters and less

TABLE III  
PRETRAINING DATASET

| LoRa Dataset in [29] |                  |             | $\mathcal{D}_p$   |          |
|----------------------|------------------|-------------|-------------------|----------|
| Rx                   | DUT              | DUT Indexes | Subset            | # of DUT |
| Pluto                | Lopy4            | 1 - 5       | $\mathcal{D}_p^1$ | 3        |
| B200 mini            | Lopy4            | 6 - 10      | $\mathcal{D}_p^2$ | 8        |
|                      | mbed 1272 shield | 1 - 5       |                   |          |
| B200                 | Lopy4            | 11 - 15     | $\mathcal{D}_p^3$ | 14       |
|                      | mbed 1272 shield | 6 - 10      |                   |          |
|                      | mbed 1261 shield | 1 - 5       |                   |          |
| B210                 | Lopy4            | 16 - 20     | $\mathcal{D}_p^4$ | 17       |
|                      | mbed 1272 shield | 11 - 15     |                   |          |
|                      | mbed 1261 shield | 6 - 10      |                   |          |
|                      | Fipy             | 1 - 5       |                   |          |

than 54 MB of storage. In contrast, methods utilizing the IQ representation are substantially more resource-intensive.

#### F. Summary

According to the above experimental evaluation, we can conclude that using the spectrogram demonstrated more consistent and higher classification accuracies, compared to the IQ samples. In addition, by adopting the Siamese network architecture, the classification performance can be further boosted. For example, as can be observed in Figs. 12 and 13, *Spec + Siamese* achieved about 10% gain over *Spec*. Therefore, *Spec + Siamese* is recommended and will be used for evaluating pretraining in Section VIII.

### VIII. EXPERIMENTAL EVALUATION OF PRETRAINING

In this section, we used a pretraining dataset to train the feature extractor and then fine-tune the classification neural network, as shown in Fig. 3. We employed the *Spec + Siamese* approach, as recommended in Section VII. This three-stage approach is denoted as *w/ pretrain*. We also used training the *Spec + Siamese* from scratch as the benchmark, denoted as *w/o pretrain*, which is the two-stage approach evaluated in Section VII.

Similar to Section VII, we also did ablation studies for the effect of pretraining against channel effect (Section VIII-B) and receiver effect (Section VIII-C). We then investigated the scenarios when both the channel and receiver effects were present in Sections VIII-D and VIII-E. For each evaluation, we carried out 4 rounds. The classification accuracy is presented in terms of the error bar.

#### A. Setup for Pretraining

The datasets introduced in Section VI-A were also used in this section as the fine-tuning dataset. In addition, the feature extractor needs to be pretrained, and the dataset as well as the configuration will be introduced below.

1) *Dataset for Pretraining*: The work in [29] released a public LoRa dataset involving various types of DUTs and SDR receivers, detailed in Table III. We used parts of the datasets in [29] to build our pretraining dataset. In particular, we considered the signals from each receiver as a subdataset.

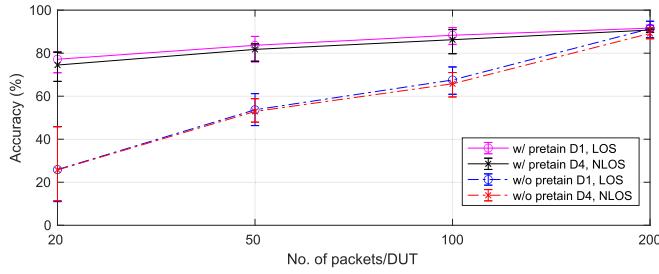


Fig. 14. Evaluation of pretraining for mitigating channel effects. *Public Dataset 1* [35] was used. Test datasets D1 (LoS) and D4 (NLoS) were used.

We randomly selected a few DUTs from each subdataset to emulate real scenarios. Specifically, the numbers of DUTs from the four receivers/subdatasets are 3/8/14/17, respectively. The number of packets obtained from each DUT ranges from 500 to 800.

2) *Architecture of Feature Extractor*: The feature extractor in Fig. 5, which has been described in Section IV-D.1, was trained in the pretraining stage.

3) *Pretraining Configuration*: The parameters of the neural network were optimized by the Adam optimizer with an initial learning rate of 0.001 and a batch size of 32. The configurations for the learning rate scheduler and early stopping were identical to those of Section VI-C.

### B. Evaluation Against Channel Effect

The *Public Dataset 1* [35] in Section VI-A.2 was used for stages 2 and 3. In particular, two test datasets, D1 and D4, were selected, representing LOS and NLOS conditions, respectively.

The results are given in Fig. 14. It can be observed that the models trained with the pretraining (*w/ pretrain*) show better performance than the models trained from scratch (*w/o pretrain*), in both LOS and NLOS scenarios. The improvement in classification accuracy from the pretrained model is particularly evident when the number of data packets used for training is small. In the LOS scenario D1, the utilization of a pretrained feature extractor results in an improvement in classification accuracy from 25.9% to 77.2%, when employing 20 packets per DUT for fine-tuning. The improvement provided by the pretrained model gradually decreases when the number of fine-tuning data packets increases. Both approaches achieved almost the same accuracies for LOS and NLOS scenarios when 200 fine-tuning data packets per DUT were used.

### C. Evaluation Against Receiver Effect

The *Public Dataset 3* [38] in Section VI-A.4 was used. The training datasets from B200-1 and B210-1 were used for fine-tuning. Datasets collected by RTL-1, Pluto-1, and N210-1 were chosen to form the test set for evaluation.

The impact of pretrained models on the receiver mitigation is presented in Fig. 15. When there were only 20 packets per DUT-receiver pair, the *w/ pretrain* achieves an accuracy improvement of 10% to 15%, compared to *w/o pretrain*. Taking RTL-1 as an example, as shown in Fig. 15(a), the average accuracy of *w/o pretrain* is 74.5%, while *w/ pretrain*

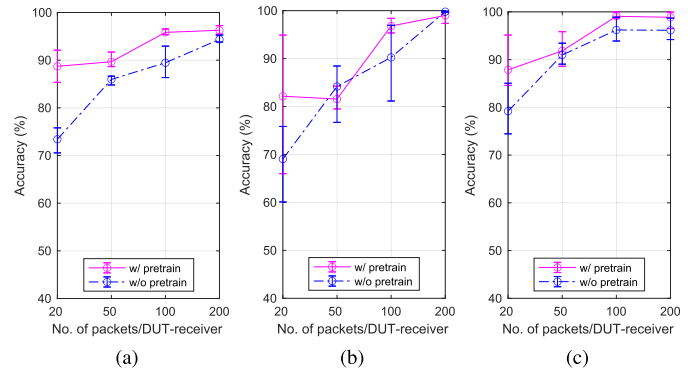


Fig. 15. Evaluation of pretrain on mitigating receiver effects. *Public Dataset 3* [38] was used. Training dataset from B200-1 and B210-1. Test datasets from static channels, half a meter between DUT and receiver. (a) Test dataset from RTL-1. (b) Test dataset from Pluto-1. (c) Test dataset from N210-1.

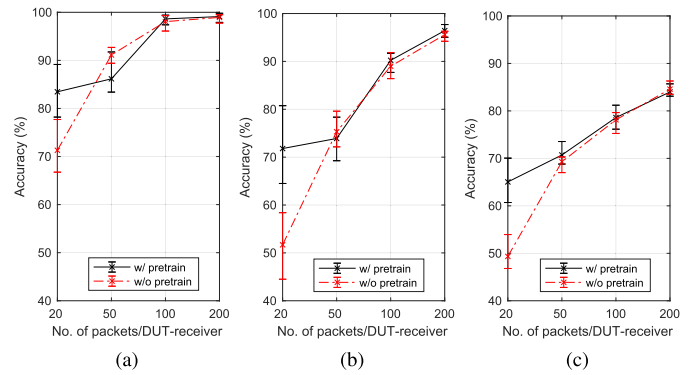


Fig. 16. Evaluation of pretrain on mitigating static channel and receiver effects. *Public Dataset 3* [38] was used. Training dataset from B200-1 and B210-1. Test datasets from static channels (NLOS) and collected by N210-1. (a) Test dataset from Location A. (b) Test dataset from Location B. (c) Test dataset from Location C.

can achieve 88.7%. Therefore, pretraining is highly useful in getting a well-performing RFFI system when fine-tuning the network with few packets.

It can also be observed that as the number of training data packets increases, the impact of whether the pre-model is utilized gradually diminishes. When there were 200 packets per DUT-receiver pair, the results of *w/ pretrain* and *w/o pretrain* became nearly identical. This phenomenon exhibits consistency across multiple SDR platforms.

### D. Evaluation Against Static Channel and Receiver Effects

The *Public Dataset 3* [38] in Section VI-A.4 was used. The training datasets from B200-1 and B210-1 were used for fine-tuning. A subset from **test dataset B**, namely the data collected from Locations A, B and C using N210-1, was selected for evaluation.

As can be observed in Fig. 16, *w/ pretrain* consistently achieves better performance across all channel environments when the number of fine-tuning samples was 20 per DUT-receiver, no matter Location A/B/C. As shown in Fig. 16(b), the accuracy in Location B improved by about 20% with the pretrained model when only 20 packets were available for fine-tuning.

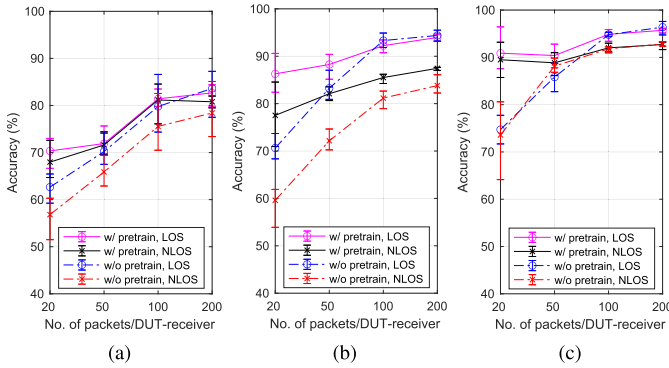


Fig. 17. Evaluation of pretrain on mitigating dynamic channel and receiver effects. The *Self-Collected Dataset* was used. Training dataset from B210-a and N210-c. Test datasets from various receivers and dynamic channels. (a) Test dataset from RTL-a. (b) Test dataset from B210-a. (c) Test dataset from N210-a.

#### E. Evaluation Against Dynamic Channel and Receiver Effects

The *Self-Collected Dataset* in Section VI-A.5 was used. The training datasets from B210-a and N210-c were employed for fine-tuning. The test dataset involving dynamic channel effects and receivers was used, with RTL-a, B210-a and N210-a chosen as examples.

Fig. 17 shows experimental results in dynamic scenarios across multiple receivers. The accuracy of all the receivers has been improved, both in LOS and NLOS, when the *w/ pretrain* approach was used. It can be found that across RTL-a, B210-a, and N210-a, *w/ pretrain* boosted the accuracy by over 8% on average when the number of training packets was 20 from each DUT-receiver pair. More specifically, taking B210-a as an example, the classification accuracy increases to 86.3%, surpassing the accuracy of 70.6% by *w/o pretrain*.

Fig. 17(c) illustrates that our proposed method enables the N210 to achieve above 90% accuracy in dynamic NLOS scenarios when there is provided with only 20 packets per DUT-receiver pair. It also can be noted that, despite utilizing the same training packets, the accuracy of RTL is consistently lower than that of B210 and N210 in most cases. RTL is a low-cost receiver and its hardware characteristics are more variable, making it harder to eliminate its inherent characteristics. Nonetheless, utilizing our proposed method, the classification accuracies of RTL-a/b are still improved. When provided with 100 training packets per DUT-receiver pair, RTL-a achieves an accuracy of approximately 81.2% in dynamic NLOS scenarios, demonstrating a 5.6% increase compared to *w/o pretrain*.

#### F. Complexity of the Three-Stage Approach

Table IV presents the computational and time complexity of the proposed three-stage approach, comprising the pretraining phase for the feature extractor, the finetuning stage, and the inference stage. The pretrain stage is computationally intensive but only needs to be performed once. The “Finetune” stage shows a range for both time ( $5 \times 10^4$  to  $7.43 \times 10^5$  ms) and epochs (68 to 129), which corresponds to different training scenarios with varying packet-per-device configurations,

TABLE IV  
COMPUTATIONAL COMPLEXITY AND TIME COMPLEXITY OF THE THREE-STAGE APPROACH

| Three-stage Approach | Time (ms)                             | Epochs | Parameter (M) | Storage (M) |
|----------------------|---------------------------------------|--------|---------------|-------------|
| Pretrain             | $1.24 \times 10^7$                    | 175    | 13.93         | 53.19       |
| Finetune             | $5 \times 10^4 \sim 7.43 \times 10^5$ | 68~129 | 13.94         | 53.19       |
| Inference            | 1.13                                  | \      | \             | \           |

specifically from 20 to 200 packets per device. The inference stage is highly efficient, requiring only 1.13 ms per sample.

#### G. Summary

In general, *w/ pretrain* has a better performance compared to *w/o pretrain*, in terms of mean and variance of the classification accuracy. The benefits of *w/ pretrain* are more significant when there are less packets for fine-tuning/training, because *w/ pretrain* has a feature extractor pretrained. In addition, the classification results exhibit significant fluctuations when the number of data packets is limited, characterized by the high error bar values. As the number of training data packets increases, the results become more stable, and the error bar values for each receiver decrease accordingly. For example, Fig. 17(b) shows that the error bar value for B210-a under the LOS scenario is 8.2% with 20 training packets for each DUT-receiver pair, and it reduces to 1.5% when increased to 200 per DUT-receiver pair.

#### IX. CONCLUSION

In this paper, we proposed a three-stage RFFI approach involving a pretraining stage enhanced by contrastive learning and a Siamese network-based RFFI classification network to mitigate channel and receiver effects simultaneously. Firstly, spectrogram typically exhibits more stable and higher accuracy compared to IQ samples, which was adopted as the signal representation. Secondly, the Siamese network with contrastive loss was designed to mitigate performance degradation caused by varying channel effects and different receiver impairments. Finally, a pretraining approach using contrastive learning was proposed to address scenarios where only a limited number of labelled packets are available for training. We carried out extensive experimental evaluations using four different LoRa datasets, including three public LoRa datasets and one self-collected dataset. With only 20 training packets per device, the accuracy can be significantly improved from 25.9% to 77.2% when utilizing a pretrained model. The proposed RFFI system achieved a classification accuracy exceeding 90% in distinguishing 10 LoRa devices across dynamic channels and various receivers. This paper considered LoRa-RFFI as a case study. However, our proposed three-stage approach is applicable to other wireless protocols. A new signal representation may need to be designed to expose signal features.

#### REFERENCES

- [1] J. Ma, J. Zhang, G. Shen, L. Peng, and A. Marshall, “Towards channel-robust radio frequency fingerprint identification using contrastive learning,” in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Mar. 2025, pp. 1–6.

- [2] J. Zhang, F. Ardizzone, M. Piana, G. Shen, and S. Tomasin, "Physical layer-based device fingerprinting for wireless security: From theory to practice," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 5296–5325, 2025.
- [3] J. Zhang, G. Li, A. Marshall, A. Hu, and L. Hanzo, "A new frontier for IoT security emerging from three decades of key generation relying on wireless channels," *IEEE Access*, vol. 8, pp. 138406–138446, 2020.
- [4] J. Zhang, R. Woods, M. Sandell, M. Valkama, A. Marshall, and J. Cavallaro, "Radio frequency fingerprint identification for narrowband systems, modelling and classification," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3974–3987, 2021.
- [5] Z. Yao et al., "A novel radio frequency fingerprint concealment method based on IQ imbalance compensation and digital pre-distortion," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7349–7361, 2024.
- [6] Q. Jiang and J. Sha, "RF fingerprinting identification in low SNR scenarios for automatic identification system," *IEEE Trans. Wireless Commun.*, vol. 23, no. 3, pp. 2070–2081, Mar. 2024.
- [7] A. Elmaghbbub and B. Hamdaoui, "Distinguishable IQ feature representation for domain-adaptation learning of WiFi device fingerprints," *IEEE Trans. Mach. Learn. Commun. Netw.*, vol. 2, pp. 1404–1423, 2024.
- [8] Y. Li, Y. Ding, J. Zhang, G. Goussetis, and S. K. Podilchak, "Radio frequency fingerprinting exploiting non-linear memory effect," *IEEE Trans. Cognit. Commun. Netw.*, vol. 8, no. 4, pp. 1618–1631, Dec. 2022.
- [9] S. Hanna, S. Karunaratne, and D. Cabric, "Open set wireless transmitter authorization: Deep learning approaches and dataset considerations," *IEEE Trans. Cognit. Commun. Netw.*, vol. 7, no. 1, pp. 59–72, Mar. 2021.
- [10] K. Sankhe et al., "No radio left behind: Radio fingerprinting through deep learning of physical-layer hardware impairments," *IEEE Trans. Cognit. Commun. Netw.*, vol. 6, no. 1, pp. 165–178, Mar. 2020.
- [11] W. He, W. Wu, X. Gu, and Z. Chen, "Diff-ADF: Differential adjacent-frame radio frequency fingerprinting for LoRa devices," in *Proc. IEEE Conf. Comput. Commun.*, Vancouver, BC, Canada, May 2024, pp. 2089–2098.
- [12] X. Yang and D. Li, "LED-RFF: LTE DMRS-based channel robust radio frequency fingerprint identification scheme," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1855–1869, 2024.
- [13] Q. Pan, Z. An, X. Yang, X. Zhao, and L. Yang, "RF-DNA: Large-scale physical-layer identifications of RFIDs via dual natural attributes," in *Proc. 28th Annu. Int. Conf. Mobile Comput. Netw. (MobiCom)*, Oct. 2022, pp. 419–431.
- [14] G. Shen, J. Zhang, A. Marshall, M. Valkama, and J. R. Cavallaro, "Toward length-versatile and noise-robust radio frequency fingerprint identification," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2355–2367, 2023.
- [15] R. Xie et al., "A generalizable model-and-data driven approach for open-set RFF authentication," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 4435–4450, 2021.
- [16] Y. Qian, J. Qi, X. Kuai, G. Han, H. Sun, and S. Hong, "Specific emitter identification based on multi-level sparse representation in automatic identification system," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2872–2884, 2021.
- [17] S. Rajendran and Z. Sun, "RF impairment model-based IoT physical-layer identification for enhanced domain generalization," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 1285–1299, 2022.
- [18] W. Wang and L. Gan, "Radio frequency fingerprinting improved by statistical noise reduction," *IEEE Trans. Cognit. Commun. Netw.*, vol. 8, no. 3, pp. 1444–1452, Sep. 2022.
- [19] J. He, S. Huang, Z. Yang, K. Yu, H. Huan, and Z. Feng, "Channel-agnostic radio frequency fingerprint identification using spectral quotient constellation errors," *IEEE Trans. Wireless Commun.*, vol. 23, no. 1, pp. 158–170, Jan. 2024.
- [20] J. He, S. Huang, Y. Chen, S. Chang, Y. Zhang, and Z. Feng, "Radio frequency fingerprint identification for OFDM system considering unknown multipath fading channel," *IEEE Trans. Cognit. Commun. Netw.*, vol. 10, no. 6, pp. 2076–2087, Dec. 2024.
- [21] P. Tang, G. Ding, Y. Xu, Y. Jiao, Y. Song, and G. Wei, "Causal learning for robust specific emitter identification over unknown channel statistics," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5316–5329, 2024.
- [22] X. Qi, A. Hu, and Z. Zhang, "Data-and-channel-independent radio frequency fingerprint extraction for LTE-V2X," *IEEE Trans. Cognit. Commun. Netw.*, vol. 10, no. 3, pp. 905–919, Jun. 2024.
- [23] P. Yin et al., "Multi-channel CNN-based open-set RF fingerprint identification for LTE devices," *IEEE Trans. Cogn. Commun. Netw.*, vol. 10, no. 5, pp. 1788–1800, 2024.
- [24] Y. Zhang, Q. Zhang, H. Zhao, Y. Lin, G. Gui, and H. Sari, "Multisource heterogeneous specific emitter identification using attention mechanism-based RFF fusion method," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 2639–2650, 2024.
- [25] L. Peng, Z. Wu, J. Zhang, M. Liu, H. Fu, and A. Hu, "Hybrid RFF identification for LTE using wavelet coefficient graph and differential spectrum," *IEEE Trans. Veh. Technol.*, vol. 73, no. 8, pp. 11621–11636, Aug. 2024.
- [26] D. Li, X. Yang, A. Hu, F. Zhou, and O. A. Dobre, "LTE device radio frequency fingerprints blind extraction based on temporal-frequency domain PRACH signals," *IEEE Trans. Veh. Technol.*, vol. 72, no. 10, pp. 13229–13242, Oct. 2023.
- [27] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1597–1607.
- [28] Z. Zhou, G. Li, T. Wang, D. Zeng, X. Li, and Q. Wang, "A robust open-set specific emitter identification for complex signals with class-irrelevant features," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 6058–6073, 2025.
- [29] G. Shen, J. Zhang, X. Wang, and S. Mao, "Federated radio frequency fingerprint identification powered by unsupervised contrastive learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 9204–9215, 2024.
- [30] L. Peng, H. Peng, H. Fu, and M. Liu, "Channel-robust radio frequency fingerprint identification for cellular uplink LTE devices," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 17154–17169, May 2024.
- [31] Y. Wang, T. Ohtsuki, Z. Sun, D. Niyato, X. Wang, and G. Gui, "Avoiding shortcuts: Enhancing channel-robust specific emitter identification via single-source domain generalization," *IEEE Trans. Wireless Commun.*, vol. 24, no. 4, pp. 3163–3176, Apr. 2025.
- [32] K. Li, J. Bao, X. Xie, J. Hong, and C. Hua, "Receiver-agnostic radio frequency fingerprint identification for zero-trust wireless networks," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 6, pp. 1981–1997, Jun. 2025.
- [33] T. Zhao, S. Sarkar, E. Krijestorac, and D. Cabric, "GAN-RXA: A practical scalable solution to receiver-agnostic transmitter fingerprinting," *IEEE Trans. Cognit. Commun. Netw.*, vol. 10, no. 2, pp. 403–416, Apr. 2024.
- [34] L. Xu et al., "Enhanced few-shot specific emitter identification via phase shift prediction and decoupling," *IEEE Trans. Cognit. Commun. Netw.*, vol. 11, no. 1, pp. 145–155, Feb. 2025.
- [35] G. Shen, J. Zhang, A. Marshall, and J. R. Cavallaro, "Towards scalable and channel-robust radio frequency fingerprint identification for LoRa," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 774–787, 2022.
- [36] A. Al-Shawabka et al., "Exposing the fingerprint: Dissecting the impact of the wireless channel on radio fingerprinting," in *Proc. IEEE Int. Conf. Comput. Commun. (INFOCOM)*, May 2020, pp. 646–655.
- [37] N. Soltani, K. Sankhe, J. Dy, S. Ioannidis, and K. Chowdhury, "More is better: Data augmentation for channel-resilient RF fingerprinting," *IEEE Commun. Mag.*, vol. 58, no. 10, pp. 66–72, Oct. 2020.
- [38] G. Shen, J. Zhang, A. Marshall, R. Woods, J. Cavallaro, and L. Chen, "Towards receiver-agnostic and collaborative radio frequency fingerprint identification," *IEEE Trans. Mobile Comput.*, vol. 23, no. 7, pp. 7618–7634, Jul. 2024.
- [39] X. Zha, T. Li, Z. Qiu, and F. Li, "Cross-receiver radio frequency fingerprint identification based on contrastive learning and subdomain adaptation," *IEEE Signal Process. Lett.*, vol. 30, pp. 70–74, 2023.
- [40] Y. Zhang, Q. Li, H. Liu, L. Yang, and J. Yang, "Domain generalization for cross-receiver radio frequency fingerprint identification," *IEEE Internet of Things J.*, vol. 12, no. 5, pp. 5207–5218, 2025.
- [41] A. Elmaghbbub and B. Hamdaoui, "LoRa device fingerprinting in the wild: Disclosing RF data-driven fingerprint sensitivity to deployment variability," *IEEE Access*, vol. 9, pp. 142893–142909, 2021.
- [42] L. Xie, L. Peng, and J. Zhang, "Towards robust RF fingerprint identification using spectral regrowth and carrier frequency offset," in *Proc. IEEE Int. Conf. Comput. Commun. (INFOCOM)*, May 2025, pp. 1–10.
- [43] A. Al-Shawabka, P. Pietraski, S. B. Pattar, F. Restuccia, and T. Melodia, "DeepLoRa: Fingerprinting LoRa devices at scale through deep learning and data augmentation," in *Proc. Int. Symp. Theory, Algorithmic Found., Protocol Design Mobile Netw. Mobile Comput.*, Shanghai, China, Jul. 2021, pp. 251–260.